

Project 3 最终版 Write-up

1. 项目概述

本项目实现了一个单核动态内存分配器，接口包括 `my_init`、`my_malloc`、`my_free`、`my_realloc` 和 `my_check`。分配器运行在项目提供的模拟堆 `memlib` 上，只能通过 `mem_sbrk` 扩展堆空间，不能直接调用 `libc` 的 `malloc/free/realloc`。驱动程序 `mdriver` 会从 `trace` 文件中读取分配、释放、重分配和写入操作，并从正确性、空间利用率和吞吐量三个角度进行评估。

最终实现位于 `mymalloc/allocator.c`，验证器位于 `mymalloc/validator.h`。当前版本采用分离空闲链表、有限最佳适应、边界标记、`prev_alloc` 位、只为空闲块保留 `footer`、立即合并和针对 `realloc` 的原地扩展优化。该实现通过了仓库中全部 `short_traces`、`traces`、`additional_traces` 和 `my_traces`，共 42 条 `trace` 的正确性验证和逐操作堆检查。

评分函数中利用率和吞吐量权重各为 0.5：

```
1 | score = 100 * sqrt(utilization * throughput_ratio)
```

因此本项目的目标不是单纯追求最快或最省空间，而是在搜索成本、内部碎片、外部碎片和元数据开销之间取得平衡。

2. 实现过程

2.1 基础正确性版本

项目初期先完成一个以正确性为核心的基础版本。该版本使用显式空闲双向链表管理所有空闲块，分配时采用首次适应策略，释放时立即合并相邻空闲块，找到过大的空闲块时按最小块大小进行分割。

基础版本的块结构如下：

```
1 | 已分配块：[header][payload][footer]
2 | 空闲块：  [header][next][prev][...][footer]
```

其中 `header` 和 `footer` 保存块大小和分配状态。空闲块 `payload` 的前两个机器字分别保存 `next` 和 `prev` 指针，因此一个可管理的空闲块至少需要 32B：

```
1 | 8B header + 8B next + 8B prev + 8B footer = 32B
```

这个阶段主要解决三个问题：

1. `free` 后的空间必须能够被后续 `malloc` 复用。
2. 相邻空闲块必须合并，避免产生连续空闲块。
3. `realloc` 必须保存旧数据，满足 `libc` 语义。

基础版本能够保证正确性，但所有空闲块集中在一个链表中，分配时需要线性搜索；同时每个已分配块也保留 `footer`，空间开销偏高。

2.2 最终版优化方向

最终版围绕两个瓶颈继续优化：

- 1. 降低搜索成本：将单一空闲链表改为分离空闲链表，让不同大小的空闲块进入不同桶。
- 2. 降低元数据开销：在 header 中加入 `prev_alloc` 位，使已分配块不再需要 footer。

最终版块结构如下：

```
1 | 已分配块: [header: size | alloc | prev_alloc][payload]
2 | 空闲块:   [header: size | alloc | prev_alloc][next][prev][...][footer]
```

堆两端使用哨兵块统一边界情况：

```
1 | [8B prologue, allocated][普通块...][0B epilogue, allocated]
```

prologue 避免处理“第一块没有前驱”的特殊情况，epilogue 标记堆尾。这样合并、扩堆和检查器逻辑都可以沿用同一套路径。

3. 核心数据结构

3.1 分离空闲链表

最终实现维护 `free_lists[MAX_CLASSES]`，其中 `MAX_CLASSES = 28`，默认有效桶数 `NUM_CLASSES = 21`。 `active_classes()` 决定实际使用多少个桶，因此 OpenTuner 可以在不修改数组结构的前提下调节桶数量。

默认大小分类边界如下：

桶号	上限
0	32B
1	40B
2	48B
3	56B
4	64B
5	72B
6	80B
7	88B
8	96B
9	112B
10	128B
11	160B
12	192B
13	256B
14	320B
15	448B
16	640B
17	896B
18	1280B
19	1792B
20	2560B 及以上

当前默认只使用前 21 个桶，因此大于 2560B 的块统一进入最后一个有效桶。数组中预留到 28 个桶，是为了给自动调参保留空间。

3.2 全局数据大小

项目限制全局和静态数据结构总大小不超过 512B。当前实现中主要静态数据为：

数据结构	大小估算
<code>free_lists[28]</code>	28 个指针，约 224B
<code>class_index</code> 中的 <code>limits[28]</code>	28 个 <code>size_t</code> ，约 224B
合计	约 448B

因此当前实现满足 512B 的静态数据限制。

4. 关键算法设计

4.1 初始化

`my_init` 会清空所有空闲链表头，然后通过 `mem_sbrk(16)` 建立 `prologue` 和 `epilogue`。初始化阶段不立即申请大空闲块，而是在第一次真实分配找不到合适块时再扩展堆。

```
1 | heap start → [prologue: 8B allocated][epilogue: 0B allocated]
```

这种延迟扩堆策略减少了短 `trace` 中不必要的堆空间占用。

4.2 分配流程

`my_malloc(size)` 的流程如下：

1. 若 `size == 0`，返回 `NULL`。
2. 通过 `adjusted_size` 将用户请求转换为真实块大小：

```
1 | asize = max(align8(size + 8B header), 32B)
```

3. 调用 `find_fit(asize)` 从对应大小桶开始搜索。
4. 如果没有找到合适空闲块，则调用 `extend_heap(max(asize, CHUNK_SIZE))` 扩展堆。
5. 调用 `place_block` 放置块，必要时分割剩余空间。
6. 返回 `payload` 指针，即 `header + 8B`。

默认 `CHUNK_SIZE = 2048`。相比传统的 4096B 扩堆粒度，2KB 对短 `trace` 和中小负载更节省空间，同时仍能避免过于频繁地调用 `mem_sbrk`。

4.3 有限最佳适应搜索

`find_fit` 不是完整最佳适应，而是有限最佳适应：

1. 先根据请求大小定位桶。
2. 从该桶开始向更大桶搜索。
3. 每个桶最多检查 `SEARCH_LIMIT = 21` 个节点。
4. 如果遇到大小完全匹配的块，立即返回。
5. 否则返回已检查范围内最小的可用块。

该策略在首次适应和完整最佳适应之间折中。首次适应速度快但容易留下碎片；完整最佳适应碎片更少但搜索成本高。有限最佳适应把搜索上限控制在较小范围内，在吞吐量和利用率之间取得较稳定的结果。

4.4 分割与放置

`place_block` 会先从空闲链表中删除选中的空闲块。如果剩余空间同时满足：

```
1 remainder ≥ MIN_BLOCK
2 remainder ≥ SPLIT_THRESHOLD
```

则分割出一个新的空闲块。默认 `MIN_BLOCK = 32`，`SPLIT_THRESHOLD = 32`。保留 `SPLIT_THRESHOLD` 是为了自动调参：当该值被调大时，可以主动避免产生较小且不容易复用的空闲块。

分割出的剩余块不会简单插回链表，而是调用 `coalesce`。这样可以处理“剩余块后面本来就是空闲块”的情况，避免形成连续空闲块。

4.5 释放与合并

`my_free(ptr)` 会将 `payload` 指针回退 8B 得到块头，把该块标记为空闲，写入 `footer`，更新后继块的 `prev_alloc` 位，然后调用 `coalesce`。

`coalesce` 统一处理四种情况：

前块状态	后块状态	处理方式
已分配	已分配	当前块直接插入空闲链表
空闲	已分配	删除前块并向前合并
已分配	空闲	删除后块并向后合并
空闲	空闲	删除前后空闲块并三块合并

合并前必须先把相邻空闲块从对应桶中删除，合并后重新写入 `header/footer`，更新下一块的 `prev_alloc`，再将新大块按大小插入对应桶。

4.6 `prev_alloc` 位与 `footer` 优化

最终版 `header` 使用低位保存状态：

```
1 bit 0: alloc
2 bit 1: prev_alloc
3 高位: block size
```

由于块大小始终按 8B 对齐，低 3 位本来就是 0，可以用于保存状态位。`prev_alloc` 表示物理前驱块是否已分配。这样判断前块是否可合并时，不必先读取前块 `footer`。

更重要的是，已分配块不再需要 `footer`。只有当前块发现前块空闲时，才需要通过前块 `footer` 反向定位前块起点；如果前块已分配，则不会参与合并，也就不需要 `footer`。这一优化使每个已分配块少 8B 元数据，提升了空间利用率并减少了不必要的内存访问。

4.7 `realloc` 优化

`my_realloc` 的处理顺序如下：

- `ptr == NULL`：等价于 `my_malloc(size)`。
- `size == 0`：等价于 `my_free(ptr)` 并返回 `NULL`。
- 当前块已经足够大：原地返回，剩余空间足够时切出空闲块。

- 4. 前一物理块空闲：尝试与前块合并；如果还不够，再尝试同时吞掉后块。
- 5. 后一物理块空闲：尝试向后原地扩展。
- 6. 仍然失败：分配新块，拷贝旧数据，释放旧块。

向后扩展时 payload 起点不变，可以零拷贝完成。向前合并时 payload 会移动到更低地址，源区域和目标区域可能重叠，因此必须使用 memmove 而不是 memcpy 。

当前实现还引入 REALLOC_EXTRA = 207 。它不是普通 malloc 的额外开销，也不是每次 realloc 都固定多申请 207B。它只在 realloc 通过相邻空闲块原地合并成功后生效：如果合并后的空间足够，则优先保留一段增长余量，减少后续渐进扩容反复搬迁的概率；如果空间不够，则按精确需求切分或占用整个合并块。

4.8 自动调参接口

实现中暴露了多个编译期宏，便于 OpenTuner 搜索参数：

参数	默认值	作用
ALIGNMENT	8	对齐粒度
NUM_CLASSES	21	有效分离桶数量
SEARCH_LIMIT	21	每个桶最多检查节点数
CHUNK_SIZE	2048	最小扩堆大小
SPLIT_THRESHOLD	32	分割阈值
REALLOC_EXTRA	207	原地 realloc 的预留增长空间

最终参数选择兼顾了 traces 和 additional_traces ，避免只对单个 trace 过拟合。

5. 正确性验证

正确性验证分为外部黑盒验证和内部堆检查两层。

5.1 外部验证器

validator.h 将分配器当作黑盒 API 检查，主要验证：

- 1. 返回的 payload 地址必须 8B 对齐。
- 2. payload 范围必须位于模拟堆内部。
- 3. 不同已分配块的 payload 不能重叠。
- 4. realloc 后必须保留 min(old_size, new_size) 字节旧数据。

验证器为每个活跃块维护一个 range_t 链表。分配时加入新区间，释放时移除区间，重分配时先移除旧区间再加入新区间。为了检查 realloc 是否正确保存数据，验证器会用 pointer id 生成特征字节填充 payload，并在后续 realloc 后逐字节检查。

5.2 内部堆检查器

my_check 检查分配器内部不变量，主要包括：

1. prologue 和 epilogue 是否正确。
2. 每个普通块是否在堆范围内。
3. 块大小是否至少 32B 且按 8B 对齐。
4. 每个块 header 中的 `prev_alloc` 是否与真实前块状态一致。
5. 空闲块 header 和 footer 是否一致。
6. 堆中是否存在连续空闲块。
7. 空闲链表节点是否真的空闲、是否位于正确桶中。
8. 双向链表的 `next/prev` 指针是否互相一致。
9. 堆遍历得到的空闲块数量是否等于空闲链表中的节点数量。

在实验中使用 `./mdriver -c` 开启逐操作检查，因此每次 `trace` 操作后都会调用 `my_check`。这比只在 `trace` 结束时检查更容易定位破坏堆结构的 bug。

6. 实验设置

实验日期：2026-06-22

实验目录：

```
1 | /Users/claerlocus/Documents/PESS/project3/mymalloc
```

实验命令：

```
1 | ./mdriver -t short_traces -c -g -v
2 | ./mdriver -t traces -c -g -v
3 | ./mdriver -t additional_traces -c -g -v
4 | ./mdriver -t my_traces -c -g -v
```

参数说明：

参数	含义
<code>-t <dir></code>	指定 <code>trace</code> 目录
<code>-c</code>	每次操作后运行堆检查器
<code>-g</code>	输出 autograder 风格的 <code>correct</code> 和 <code>perfidx</code>
<code>-v</code>	输出每条 <code>trace</code> 的详细结果

计时方式来自 `config.h`，当前使用 `gettimeofday()`。吞吐量结果会受到机器、系统 `malloc`、编译优化参数和计时噪声影响，因此利用率比吞吐量更稳定。

7. 实验运行结果

7.1 总体结果

Trace 目录	正确数	checked	几何平均利用率	几何平均吞吐量比值	perfidx
short_traces/	2/2	全部通过	100.000000%	95.903301%	97.930231
traces/	10/10	全部通过	83.365071%	100.000000%	91.304475
additional_traces/	10/10	全部通过	80.806422%	99.969636%	89.878744
my_traces/	20/20	全部通过	100.000000%	95.524336%	97.736552

总体上，最终版在官方基础 trace 和额外 trace 中均通过正确性验证，并且吞吐量接近或达到评分上限。
my_traces 是自定义微型正确性测试，单条 trace 操作数量很少，计时容易出现 0 或无穷大，因此主要用于验证边界行为，不用于严肃性能结论。

7.2 traces/ 详细结果

Trace	valid	checked	util	ops	my Kops/sec	tput ratio
trace_c5_v0	yes	yes	97%	30610	68725	100%
trace_c1_v0	yes	yes	97%	25958	71157	100%
trace_c0_v0	yes	yes	99%	5694	57169	100%
trace_c4_v0	yes	yes	88%	18604	59495	100%
trace_c3_v0	yes	yes	60%	5768	51685	100%
trace_c7_v0	yes	yes	99%	3840	51268	100%
trace_c6_v0	yes	yes	86%	7996	23057	100%
trace_c2_v0	yes	yes	94%	4800	22274	100%
trace_c9_v0	yes	yes	49%	14401	52064	100%
trace_c8_v0	yes	yes	83%	8402	13576	100%

该组 trace 的几何平均利用率为 83.365071%，吞吐量比值达到 100.000000%，最终 perfidx = 91.304475 。

7.3 additional_traces/ 详细结果

Trace	valid	checked	util	ops	my Kops/sec	tput ratio
trace_c1_v1	yes	yes	97%	27380	54153	100%
trace_c5_v1	yes	yes	97%	30610	68802	100%
trace_c4_v1	yes	yes	87%	18072	60000	100%
trace_c0_v1	yes	yes	99%	5848	52637	100%
trace_c7_v1	yes	yes	98%	5358	62302	100%
trace_c3_v1	yes	yes	65%	8290	49581	100%
trace_c2_v1	yes	yes	93%	4800	22409	100%
trace_c6_v1	yes	yes	74%	7792	21839	100%
trace_c9_v1	yes	yes	52%	14401	63806	100%
trace_c8_v1	yes	yes	64%	6548	13138	100%

该组 trace 的几何平均利用率为 80.806422%，吞吐量比值为 99.969636%，最终 perfidx = 89.878744 。

7.4 结果分析

`traces/` 和 `additional_traces/` 中吞吐量整体较高，主要来自三个优化：

1. 分离空闲链表减少了无关空闲块扫描。
2. 每桶搜索上限避免了长链表导致的最坏情况开销。
3. LIFO 头插使释放后的热块能够很快被复用。

利用率方面，不同 `trace` 差异较大。`trace_c0`、`trace_c1`、`trace_c5`、`trace_c7` 的利用率接近或超过 97%，说明常规分配释放模式下分离链表和立即合并效果较好。`trace_c3`、`trace_c8` 和 `trace_c9` 利用率较低，原因分别包括小块/中块混合、扩堆粒度与分配序列不完全匹配、渐进式 `realloc` 导致的堆高水位和外部碎片。

其中 `trace_c9` 是最典型的特殊负载。它包含大量渐进式 `realloc`，当前策略优先保证搜索和链表操作的吞吐量，但 LIFO 插入与有限搜索不保证维持最优物理地址布局，因此会牺牲一部分空间利用率。当前实现已经通过向前合并、向后合并和双向合并减少搬迁次数，但仍有进一步优化空间。

8. 耗时表

阶段	耗时
设计	5.0 h
编码	10.0 h
调试	5.0 h
测试	3.5 h
会议	2.0 h
合计	25.5 h

9. 当前局限与改进方向

当前版本已经通过全部仓库 `trace`，并在官方 `trace` 上取得较好的综合分数，但仍存在可以改进的地方：

1. `trace_c9` 这类渐进式 `realloc` 负载下利用率仍偏低，可以考虑对堆尾块直接扩堆并保持原地址。
2. `REALLOC_EXTRA` 是固定值，后续可以根据历史增长步长自适应调整预留空间。
3. LIFO 头插有利于吞吐量，但部分负载下可能打乱地址局部性；可以尝试对大块或 `realloc` 热块使用地址有序策略。
4. 当前 `my_check` 已能检查大多数结构不变量，但没有显式环检测；如果空闲链表形成环，检查器可能无法及时报告。
5. `adjusted_size(size)` 可以增加整数溢出保护，使接口在极端输入下更健壮。

10. 总结

本项目最终实现了一个满足 `libc` 基本语义的动态内存分配器。实现过程先以显式空闲链表保证正确性，再通过分离空闲链表、有限最佳适应、`prev_alloc` 位、去除已分配块 `footer` 和 `realloc` 原地扩展优化性能。

实验结果显示，最终版通过全部 42 条 trace 的正确性验证和逐操作堆检查。在 `traces/` 中达到 83.365071% 的几何平均利用率和 100.000000% 的吞吐量比值，综合得分 91.304475；在 `additional_traces/` 中达到 80.806422% 的几何平均利用率和 99.969636% 的吞吐量比值，综合得分 89.878744。该结果说明最终版在保持正确性的基础上，较好地平衡了空间利用率与吞吐量。